n updating the existing record.

sql
```
CREATE TABLE pcirunnermachinebuilds (
    partitionid bigint DEFAULT 100 NOT NULL,
    buildid bigint NOT NULL,
    runnermachineid bigint NOT NULL
)
PARTITION BY LIST (partitionid);

CREATE TABLE cirunnermachines (
    id bigint NOT NULL,
    systemxid character varying UNIQUE NOT NULL,
    contactedat timestamp without time zone,
    version character varying,
    revision character varying,
    platform character varying,
    architecture character varying,
    ipaddress character varying,
    executortype smallint,
    config jsonb DEFAULT '{}'::jsonb NOT NULL
);
```

Advantages

- Easier for users to wrap their minds around the concept: instead of two types of tokens, there is a single type of token - the per-runner authentication token. Having two types of tokens frequently results in misunderstandings when discussing issues;
- Runners can always be traced back to the user who created it, using the audit log;
- The claims of a CI runner are known at creation time, and cannot be changed from the runner (for example, changing the accesslevel/protected flag). Authenticated users may however still edit these settings through the GitLab UI;
- Easier cleanup of stale runners, which doesn't touch the cirunner table.

Details

In the proposed approach, we create a distinct way to configure runners that is usable alongside the current registration token method during a transition period. The idea is to avoid having the Runner make API calls that allow it to leverage a single "god-like" token to register new runners.

The new workflow looks as follows:

1. The user opens the Runners settings page (instance, group, or project level);
1. The user fills in the details regarding the new desired runner, namely description, tags, protected, locked, etc.;
1. The user clicks Create. That results in the following:

1. Creates a new runner in the cirunners table (and corresponding glrt- prefixed authentication token);
1. Presents the user with instructions on how to configure this new runner on a machine, with possibilities for different supported deployment scenarios (for example, shell, docker-compose, Helm chart, etc.)
   This information contains a token which is available to the user only once, and the UI makes it clear to the user that the value shall not be shown again, as registering the same runner multiple times
   is discouraged (though not impossible).

1. The user copies and pastes the instructions for the intended deployment scenario (a register command), leading to the following actions:

1. Upon executing the new gitlab-runner register command in the instructions, gitlab-runner performs
   a call to the POST /api/v4/runners/verify with the given runner token;
1. If the POST /api/v4/runners/verify GitLab endpoint validates the token, the config.toml file is populated with the configuration;
1. Whenever a runner pings for a job, the respective cirunnermachines record is "upserted" with the latest information about the
   runner (with Redis cache in front of it like we do for Runner heartbeats).

As part of the transition period, we provide admins and top-level group owners with an instance/group-level setting (allowrunnerregistrationtoken) to disable the legacy registration token functionality and enforce using only the new workflow.
Any attempt by a gitlab-runner register command to hit the POST /api/v4/runners endpoint to register a new runner with a registration token results in a HTTP 410 Gone status code.

The instance setting is inherited by the groups. This means that if the legacy registration method
is disabled at the instance method, the descendant groups/projects mandatorily prevents the legacy
registration method.

The registration token workflow is to be deprecated (with a deprecation notice printed by the gitlab-runner register command)
and removed at a future major release after the concept is proven stable and customers have migrated to the new workflow.

Handling of legacy runners

Legacy versions of GitLab Runner do not send the unique system identifier in its requests, and we
will not change logic in Workhorse to handle unique system IDs. This can be improved upon in the
future after the legacy registration system is removed, and runners have been upgraded to newer versions.

Job pings from such legacy runners results in a cirunnermachines record containing a <legacy> systemxid field value.

Not using the unique system ID means that all connected runners with the same token are notified, instead of just the runner matching the exact system identifier. While not ideal, this is
not an issue per-se.

cirunnermachines record lifetime

New records are created in 2 situations:

- When the runner calls the POST /api/v4/runners/verify endpoint as part of the gitlab-runner register command, if the specified runner token is prefixed with glrt-. This allows the frontend to determine whether the user has successfully completed the registration and take an appropriate action;
- When GitLab is pinged for new jobs and a record matching the token+systemid does not already exist.

Due to the time-decaying nature of the cirunnermachines records, they are automatically cleaned after 7 days after the last contact from the respective runner.

Required adaptations

Migration to cirunnermachines table

When details from cirunnermachines are needed, we need to fall back to the existing fields in cirunner if a match is not found in cirunnermachines.

REST API

API endpoints receiving runner tokens should be changed to also take an optional systemid parameter, sent alongside with the runner token (most often as a JSON parameter on the request body).

GraphQL CiRunner type

The CiRunner type closely reflects the
cirunners model. This means that machine information such as ipAddress, architectureName, and executorName among others are no longer singular values in the proposed approach. We can live with that fact for the time being and start returning lists of unique values, separated
by commas.
The respective CiRunner fields must return the values for the cirunnermachines entries (falling back to cirunner record if non-existent).

Stale runner cleanup

The functionality to
clean up stale runners needs
to be adapted to clean up cirunnermachines records instead of cirunners records.

At some point after the removal of the registration token support, we'll want to create a background
migration to clean up stale runners that have been created with a registration token (leveraging the
enum column created in the cirunners table.

Runner creation through API

Automated runner creation is possible through a new GraphQL mutation and the existing POST /user/runners REST API endpoint.
These endpoints are only available to users that are
allowed to create runners at the specified
scope.

Implementation plan

Stage 1 - Deprecations

Component	Milestone	Changes
GitLab Rails app	15.6	Deprecate POST /api/v4/runners endpoint for 17.0. This hinges on a proposal to allow deprecating REST API endpoints for security reasons.
GitLab Runner	15.6	Add deprecation notice for register command for 17.0.
GitLab Runner Helm Chart	15.6	Add deprecation notice for runnerRegistrationToken command for 17.0.
GitLab Runner Operator	15.6	Add deprecation notice for runner-registration-token command for 17.0.
GitLab Runner / GitLab Rails app	15.7	Add deprecation notice for registration token reset for 17.0.

Stage 2 - Prepare gitlab-runner for systemid

Component	Milestone	Changes
GitLab Runner	15.7	Ensure a sidecar TOML file exists with a systemid value. Log new system ID values with INFO level as they get assigned.
GitLab Runner	15.9	Log unique system ID in the build logs.
GitLab Runner	15.9	Label Prometheus metrics with unique system ID.
GitLab Runner	15.8	Prepare register command to fail if runner server-side configuration options are passed together with a new glrt- token.

Stage 2a - Prepare GitLab Runner Helm Chart and GitLab Runner Operator

Component	Milestone	Changes
GitLab Runner Helm Chart	%15.10	Update the Runner Helm Chart to support registration with the authentication token.
GitLab Runner Operator	%15.10	Update the Runner Operator to support registration with the authentication token.

| GitLab Runner Helm Chart | %16.2 | Add systemID to Runner Helm Chart. |

Stage 3 - Database changes

<!-- markdownlint-disable MD056 -->

Component	Milestone	Changes
GitLab Rails app	%15.8	Create database migration to add columns to cirunners table.
GitLab Rails app	%15.8	Create database migration to add cirunnermachines table.
GitLab Rails app	%15.9	Create database migration to add cirunnermachines.id foreign key to cibuildsmetadata table.
GitLab Rails app	%15.8	Create database migrations to add allowrunnerregistrationtoken setting to applicationsettings and namespacesettings tables (default: true).
GitLab Rails app	%15.8	Create database migration to add config column to cirunnermachines table.
GitLab Runner	%15.9	Start sending systemid value in POST /jobs/request request and other follow-up requests that require identifying the unique system.
GitLab Rails app	%15.9	Create service similar to StaleGroupRunnersPruneCronWorker service to clean up cirunnermachines records instead of cirunners records. Existing service continues to exist but focuses only on legacy runners.
GitLab Rails app	%15.9	Implement the createrunnermachine feature flag.
GitLab Rails app	%15.9	Create cirunnermachines record in POST /runners/verify request if the runner token is prefixed with glrt-.
GitLab Rails app	%15.9	Use runner token + systemid JSON parameters in POST /jobs/request request in the heartbeat request to update the cirunnermachines cache/table.
GitLab Rails app	%15.9	Implement the createrunnerworkflowforadmin feature flag.
GitLab Rails app	%15.9	Implement create{instance|group|project}runner permissions.
GitLab Rails app	%15.9	Rename cirunnermachines.machinexid column to systemxid to be consistent with systemid passed in APIs.
GitLab Rails app	%15.10	Remove the ignore rule for cirunnermachines.machinexid column.
GitLab Rails app	%15.10	Replace cibuildsmetadata.runnermachineid with a new join table.
GitLab Rails app	%15.11	Drop cibuildsmetadata.runnermachineid column.
GitLab Rails app	%16.0	Remove the ignore rule for cibuildsmetadata.runnermachineid column.

<!-- markdownlint-enable MD056 -->

Stage 4 - Create runners from the UI

Component	Milestone	Changes
GitLab Rails app	%15.9	Add prefix to newly generated runner authentication tokens.
GitLab Rails app	%15.9	Add new runner field for with token that is used in registration
GitLab Rails app	%15.9	Implement new GraphQL user-authenticated API to create a new runner.
GitLab Rails app	%15.10	Return token and runner ID information from /runners/verify REST endpoint.
GitLab Runner	%15.10	Modify register command to allow new flow with glrt- prefixed authentication tokens.

GitLab Runner	%15.10	Make the gitlab-runner register command happen in a single operation.
GitLab Rails app	%15.10	Define feature flag and policies for "New Runner creation workflow" for groups and projects.
GitLab Rails app	%15.10	Only update runner contactedat and status when polled for jobs.
GitLab Rails app	%15.10	Add GraphQL type to represent runner managers under CiRunner.
GitLab Rails app	%15.11	Implement UI to create new instance runner.
GitLab Rails app	%15.11	Update service and mutation to accept groups and projects.
GitLab Rails app	%15.11	Implement UI to create new group/project runners.
GitLab Rails app	%15.11	Add runnermachine field to CiJob GraphQL type.
GitLab Rails app	%15.11	UI changes to runner details view (listing of platform, architecture, IP address, etc.) (?)
GitLab Rails app	%15.11	Adapt POST /api/v4/runners REST endpoint to accept a request from an authorized user with a scope instead of a registration token.
GitLab Runner	%15.11	Handle glrt- runner tokens in unregister command.
GitLab Runner	%15.11	Runner asks for registration token when a glrt- runner token is passed in --token.
GitLab Rails app	%15.11	Move from 'runner machine' terminology to 'runner manager'.

Stage 5 - Optional disabling of registration token

<!-- markdownlint-disable MD056 -->

| Component | Milestone | Changes
|
|---|---:|---

---|
GitLab Rails app	%16.0	Adapt register{group	project}runner permissions to take application setting in consideration.
GitLab Rails app	%16.1	Make the POST /api/v4/runners endpoint return HTTP 410 Gone permanently if either allowrunnerregistrationtoken setting disables registration tokens. The Runners API v5 should return HTTP 404 Not Found.
GitLab Rails app	%16.1	Add runner group metadata to the runner list.
GitLab Rails app	%16.11	Add UI to allow disabling use of registration tokens in top-level group settings.
GitLab Rails app	%16.11	Add UI to allow disabling use of registration tokens in admin panel.
GitLab Rails app	%16.11	Hide legacy UI showing registration with a registration token, if it disabled on in top-level group settings or by admins.

<!-- markdownlint-enable MD056 -->

Stage 6 - Enforcement

Component	Milestone	Changes

GitLab Rails app	%17.0	Disable registration tokens for all groups by running database migration (only on GitLab.com)
GitLab Rails app	%17.0	Disable registration tokens on the instance level by running database migration (except GitLab.com)
GitLab Rails app	%16.3	Implement new :createrunner PPGAT scope so that we don't require a full api scope.
GitLab Rails app		Document gotchas when automatically rotating runner tokens with multiple machines.

Stage 7 - Removals

Prior to March 2025, the removal plan called for the removal of the Runner registration token capability in GitLab 18.0, the May 2025 release. After careful consideration, we have decided not to remove the Runner Registration Token capability from GitLab in the 18.0 release - May 2025. We may revisit this in the future, so for now, we are not targeting any specific milestone for removal. We recommend that customers who still rely on the runner registration token method, discontinue the use of that method for registering new runners, and adopt the runner creation flow instead.

Component	Milestone	Changes
GitLab Rails app	N/A	Remove UI enabling registration tokens on the group and instance levels.
GitLab Rails app	N/A	Remove legacy UI showing registration with a registration token.
GitLab Runner	N/A	Remove runner model arguments from register command (for example --run-untagged, --tag-list, etc.)
GitLab Rails app	N/A	Create database migrations to drop allowrunnerregistrationtoken setting columns from applicationsettings and namespacesettings tables.
GitLab Rails app	N/A	Create database migrations to drop: - runnersregistrationtoken/runnersregistrationtokenencrypted columns from applicationsettings; - runnerstoken/runnerstokenencrypted from namespaces table; - runnerstoken/runnerstokenencrypted from projects table.
GitLab Rails app	N/A	Remove GITLABSHAREDRUNNERSREGISTRATIONTOKEN.

FAQ

Follow the user documentation.

Status

Status: RFC.

Who

Proposal:

<!-- vale gitlab.Spelling = NO -->

Role	Who
Authors	Kamil Trzci·ski, Tomasz Maczukin, Pedro Pombeiro
Architecture Evolution Coach	Kamil Trzci·ski
Engineering Leader	Nicole Williams, Cheryl Li
Product Manager	Darren Eastman, Jackie Porter
Domain Expert / Runner	Tomasz Maczukin

DRIs:

Role	Who
Leadership	Nicole Williams
Product	Darren Eastman
Engineering	Tomasz Maczukin, Pedro Pombeiro

Domain experts:

Area	Who
Domain Expert / Runner	Tomasz Maczukin

<!-- vale gitlab.Spelling = YES -->

SECTION: engineering/architecture/design-documents/runway/_index.md

<!-- Blueprints often contain forward-looking statements -->
<!-- vale gitlab.FutureTense = NO -->
{{< engineering/design-document-header >}}

Summary

Runway is an internal Platform as a Service for GitLab, which aims to enable teams to deploy and run their services quickly and safely.

Motivation

<!--
This section is for explicitly listing the motivation, goals and non-goals of
this blueprint. Describe why the change is important, all the opportunities,
and the benefits to users.

The motivation section can optionally provide links to issues that demonstrate interest in a blueprint within the wider GitLab community. Links to documentation for competing products and services is also encouraged in cases where they demonstrate clear gaps in the functionality GitLab provides.

For concrete proposals we recommend laying out goals and non-goals explicitly, but this section may be framed in terms of problem statements, challenges, or opportunities. The latter may be a more suitable framework in cases where the problem is not well-defined or design details not yet established.
-->

The underlying motivation for this initiative is covered in the Service Integration blueprint. This blueprint can be considered the implementation of the strategic requirements put forward in that proposal.

Goals

<!--
List the specific goals / opportunities of the blueprint.

- What is it trying to achieve?
- How will we know that this has succeeded?
- What are other less tangible opportunities here?
-->

- Development teams aiming to deploy their service without having to worry too much about managing the infrastructure, scaling, monitoring.
- We are focusing on satellite services that are stateless and thus can be autoscaled to meet demand.
- We aim to integrate with existing GitLab features and tooling to provide a streamlined experience.

Non-Goals

<!--
Listing non-goals helps to focus discussion and make progress. This section is optional.

- What is out of scope for this blueprint?
-->

- Hosting the GitLab monolith: The monolith is a complex application with very specialized requirements, and as such is out of scope. Deployment of the monolith is owned by the Delivery team, and there are other tools and initiatives that target this space, e.g. Cells.
- Arbitrary GCP resources: While we may support a commonly used subset of GCP resources, we will be selective with what we support. If you need more flexibility, you may want to request a GitLab Sandbox project instead.
- Arbitrary Kubernetes resources: As a managed platform, we aim not to expose too much of the underlying deployment mechanisms. This allows us to have a well-supported subset and gives us the flexibility to change providers. If you have specialized requirements, getting your own

Kubernetes cluster may be a better option.

Proposal

<!--
This is where we get down to the specifics of what the proposal actually is,
but keep it simple! This should have enough detail that reviewers can
understand exactly what you're proposing, but should not include things like
API designs or implementation. The "Design Details" section below is for the
real nitty-gritty.

You might want to consider including the pros and cons of the proposed solution so that they can be
compared with the pros and cons of alternatives.
-->

Runway is a means for deploying a service, packaged up as a Docker image to a production environment. It leverages GitLab CI/CD as well as other GitLab product features to do this.

Design and implementation details

<!--
This section should contain enough information that the specifics of your
change are understandable. This may include API specs (though not always
required) or even code snippets. If there's any ambiguity about HOW your
proposal will be implemented, this is the place to discuss them.

If you are not sure how many implementation details you should include in the
blueprint, the rule of thumb here is to provide enough context for people to
understand the proposal. As you move forward with the implementation, you may
need to add more implementation details to the blueprint, as those may become
an important context for important technical decisions made along the way. A
blueprint is also a register of such technical decisions. If a technical
decision requires additional context before it can be made, you probably should
document this context in a blueprint. If it is a small technical decision that
can be made in a merge request by an author and a maintainer, you probably do
not need to document it here. The impact a technical decision will have is
another helpful information - if a technical decision is very impactful,
documenting it, along with associated implementation details, is advisable.

If it's helpful to include workflow diagrams or any other related images.
Diagrams authored in GitLab flavored markdown are preferred. In cases where
that is not feasible, images should be placed under images/ in the same
directory as the index.md for the proposal.
-->

The design of Runway aims to decouple individual components in a way that allows them to be changed and replaced over time.

Architecture

Diagram Source

Initial Architecture Discussion

Provisioner

Provisioner is the privileged code that creates service accounts and minimal resources needed by the rest of the system.

This process is responsible for taking a request "create an experimentation space for me", and stamping out the minimum required infrastructure for that space. It also covers decommissioning when a space is no longer needed.

- It is currently based on Terraform.
- Terraform runs via CI on the provisioner project.
- We store terraform state in the GitLab Terraform state backend.

Reconciler

The Reconciler is the heart of the system. It is responsible for creating a desired view of the world (based on service definition and current version), finding the differences from the actual state, and then applying that diff.

Deploying a new version of a service is a matter of invoking the Reconciler.

This process is responsible for taking an artifact (e.g. a Docker image) from a service developer and bringing that into a runtime. This includes rollout strategies, rollbacks, canarying, multi-environment promotion, as well as diagnostic tools for failed deploys. Some of these capabilities may also be delegated to the runtime. There should also be a standard way for connecting an existing code base to a deployment.

- It is currently based on Terraform.
- Terraform runs via CI on the deployment project, triggered as a downstream pipeline from the service project.
- We store terraform state in the GitLab Terraform state backend on the deployment project.

The user-facing integration with the Reconciler is mediated via ci-tasks/service-project/runway.yml, which is a version-locked CI task that service projects include into their CI config.

Runtime

The runtime is responsible for actually scheduling and running the service workloads. Reconciler targets a runtime. Runtime will provide autoscaled compute resources with a degree of tenant isolation. It will also optionally expose an endpoint at which the workload can be reached. This endpoint will have a DNS name and be TLS encrypted.

- It is currently based on Cloud Run.

- If we need more flexibility, Knative is a likely migration target.

A service should expose an HTTP port, and it should be stateless (allowing instances to be auto-scaled). Other execution models (e.g. scheduled jobs) may be supported in the future.

Images used by Runtime

The images deployed by Runway are built by the teams responsible for the service. They are able to build the image in any fashion they wish and keep it inside the GitLab container registry of the service project. As part of the Runway deployment process, this image is mirrored to GCP Artifact Registry before being consumed by Cloud Run. This is for two reasons:

1. Cloud run is only able to consume images from GCP Artifact Registry.
1. This means that if for whatever reason the image tag is changed in the future (by error), the image running inside Runway is not affected.

GCP Project Layout

Runway currently uses shared GCP projects based off three environments (dev, staging, production). These GCP projects are

- Dev: runway-dev-527768b3 (managed by IT HackyStack)
- Staging: gitlab-runway-staging (managed by reliability
- Production: gitlab-runway-production (managed by reliability

Documents and Schemas used by Runway

In order for runway to function, there are two JSON/YAML documents in use. They are:

1. The Runway Inventory Model. This covers what service projects are currently onboarded into Runway. It's located here. The schema used to validate the docuemnt is located here. There is no backwards compatibility guarenteed to changes to this document schema. This is because it's only used internally by the Runway team, and there is only a single document actually being used by Runway to provision/deprovision Runway services.

1. The runway Service Model. This is used by Runway users to pass through configuration needed to Runway in order to deploy their service. It's located inside their Service project, at .runway/runway.yml. An example is here. The schema used to validate the document is located here. We aim to continue to make improvements and changes to the model, but all changes to the model within the same kind/apiVersion must be backwards compatible. In order to make breaking changes, a new apiVersion of the schema will be released. The overall goal is to copy the Kubernetes model for making API changes.

There are also GitLab CI templates used by Runway users in order to automate deployments via Runway through GitLab CI. Users will be encouraged to use tools such as Renovate bot in order to make sure the CI templates and
version of Runway they are using is up to date. The Runway team will support all released versions of Runway, with the exception of when a security issue is identified. When this happens, Runway users will be expected to update to a version of Runway that contains a fix for the issue as soon as possible (once notification is received).

Secrets management

For secrets management we aim to integrate with our existing HashiCorp Vault setup. We will sync secrets from Vault to whatever secrets store the Runtime integrates best with. For Cloud Run, we will use Google Secret Manager. For Kubernetes, we would use external-secrets to sync to Kubernetes Secret objects.

The following high level diagram shows the proposed setup of secrets within Vault, for consumption via runway. The general idea is a top level namespace (/runway) will be made in Vault, with roles and policies such that:

- Runway team members have full privileges over the namespace.
- The runway provisioner running in CI has the ability to create/modify/delete new service namespaces at runway/env/$environment/service. The environments currently needed are dev, staging, and production.
- The runway reconciler service accounts and GitLab team members will need read only access to runway/env/$environment/service/$runwayserviceid in order to read secrets for deployment.
- The runway reconciler will mirror secrets in Vault into Google Secrets Manager for consumption in Cloud Run via its native secrets integration.

Diagram Source: https://gitlab.com/gitlab-org/gitlab/-/blob/master/doc/architecture/blueprints/runway/img/runwayvault4.drawio

Identity Management, Authentication, Authorization across Runway Components

The goal of Runway is to not rely on long lived secrets or tokens inside the runway components themselves. In order to achieve this, all pieces of runway authenticate to each other in the following ways.

Service projects to runway deployment projects

This is handled by GitLab downstream pipeline triggers. Because of this, all permissions are handled within GitLab itself (and API calls to GitLab use short lived CIJOBTOKEN). We leverage CIJOBTOKEN allowlists to allow deployment projects and service projects to interact in API calls (e.g. updating environments in the service project).

Deployment project to GCP Cloud

GitLab CI pipelines in the deployment project are responsible for talking to GCP to provision and change the cloud resources for a Runway Service. This is done via OpenID Connnect leveraging setup done in the Runway provisioner, in order to make deployment projects authenticate as a GCP service account
with restricted permissions.

Reconciler to GCP Cloud

The reconciler (runwayctl wrapping around terraform), runs inside GitLab CI in the deployment project. This uses a specific service account setup for each Runway service, with only the

permissions it needs to manipulate the GCP apis for its work. The authentication for this is handled by GitLab CI as described above.

Observability

Observability will be covered in a separate blueprint.

Alternative Solutions

<!--
It might be a good idea to include a list of alternative solutions or paths considered,
although it is not required. Include pros and cons for
each alternative solution/path.

"Do nothing" and its pros and cons could be included in the list too.
-->

Unmanaged GCP project

Instead of building a managed platform, we could give teams a GCP project and let them have at it. In fact, this is what we have done for a few services. It brings excellent isolation and flexibility. There are however several issues with this approach:

- Missing Infrastructure-as-Code: Without any existing structure, teams may be inclined to create cloud resources via UI without using Terraform or similar IaC tools. This makes it very difficult to audit changes and re-provision the infrastructure across several environments.
- Sprawling: Without sane and safe defaults, every service will need to develop their own approach for deploying services. This also makes it very difficult for anyone else to contribute to these services.
- Non-scalable design: By being able to create arbitrary VMs, it can be tempting to co-locate services on a single machine without thinking about horizontal scalability.

Unmanaged Kubernetes cluster or namespace

Instead of building a custom platform, we could decide that Kubernetes is the platform, and let teams have either their own cluster or their own namespace in a shared cluster.

Some challenges with this approach:

- Baseline cost for GKE: The management fee is $70 per month. If we aim to have many services across multiple environments, some of them receiving low traffic volume, then a separate cluster per service is not cost effective. This suggests we would want to go with a shared cluster.
- Developer friendliness: Kubernetes is growing in popularity but it's not the most developer friendly interface. There are a lot of concepts and abstractions to grapple with. A narrower interface a la "deploy this container and give me a port", while less flexible, has a much lower barrier to entry.

It is plausible that Runway will evolve more in this direction.

GCP project per service

GCP projects provide a very solid foundation for resource isolation and cost attribution.
Ideally we would leverage such a model. However, the lack of shared resources comes at a significant baseline cost per service.

If we want to make use of GKE, the per-cluster management fee of $70 per month would likely result in the minimum cost for a service being $140 per month. For small services, this is not cost effective.

An alternative to consider would be running Kubernetes without GKE. This then means we need to manage that Kubernetes deployment ourselves, and stay on top of the differences to GKE.

Another alternative to consider is to use Cloud Run for small services and GKE for larger ones. This however requires maintaining compatibility with Cloud Run, and potentially be limited to the lowest common denominator in terms of features.

Some hybrid is possible (e.g. give a Runway service its own project), but as long as we have shared resources, we need to implement permission control on a per-resource level.

SECTION: engineering/architecture/design-documents/saas_trial_eligibility_system/_index.md

<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Summary

This design document outlines the planned implementation of a new trial eligibility system for GitLab SaaS.
The system aims to improve trial management by ensuring users can only start a trial on a namespace according to various internal business conditions,
preventing abuse of the trial system, and implementing efficient caching mechanisms for trial eligibility checks from the GitLab side.
GitLab will utilize the trial eligibility system in many scenarios:

- CTAs throughout the UI.
- Listing namespaces that a user owns that are eligible for a trial in the UI.
- Applying for a trial on a namespace from the UI.

Motivation

Currently, GitLab checks trial eligibility using GitLab database level queries.
A recent business added eligibility qualification that triggered this was solved in the short term with https://gitlab.com/gitlab-org/gitlab/-/issues/500359.
However, these queries are becoming more complex and are merely a duplication of logic that is already defined

in CustomersDot.
That issue highlighted the growing need to rely on CustomersDot for the SSOT for namespace trial eligibility.
CustomersDot already defines the eligibility conditions, and we'd like to move towards using that as the SSOT.
This also allows us to implement a solution that builds this SSOT mechanism and convert GitLab to use it in order to allow longer
term expansion of business needs in the trial eligibility space and rely on CustomersDot to be that SSOT.

reference epic: https://gitlab.com/groups/gitlab-org/-/epics/16169

Goals

- Implement a robust and accurate system for checking trial eligibility from GitLab.
- Enable easy extension of the trial eligibility rules.

Out of Scope

- Self-managed solution.
- Duo Pro/Duo Enterprise add on only trials. See https://gitlab.com/gitlab-org/gitlab/-/issues/507859note2364566118.

Proposal

The new trial eligibility system will consist of the following components:

1. CustomersDot API Endpoint: A new endpoint in the Customer Dot system to check namespace trial eligibility.
2. Caching Mechanism: Implement caching in GitLab to store trial eligibility information for a namespace.
3. Trial Application Process: Update the existing processes to use the new eligibility criteria.
4. Cache Invalidation: Implement a mechanism to invalidate the cache when necessary.

Design and implementation details

<!--
This section should contain enough information that the specifics of your
change are understandable. This may include API specs (though not always
required) or even code snippets. If there's any ambiguity about HOW your
proposal will be implemented, this is the place to discuss them.

If you are not sure how many implementation details you should include in the
document, the rule of thumb here is to provide enough context for people to
understand the proposal. As you move forward with the implementation, you may
need to add more implementation details to the document, as those may become
valuable context for important technical decisions made along the way. A
document is also a register of such technical decisions. If a technical
decision requires additional context before it can be made, you probably should

document this context in a document. If it is a small technical decision that can be made in a merge request by an author and a maintainer, you probably do not need to document it here. The impact a technical decision will have is another helpful information - if a technical decision is very impactful, documenting it, along with associated implementation details, is advisable.

If it's helpful to include workflow diagrams or any other related images. Diagrams authored in GitLab flavored markdown are preferred. In cases where that is not feasible, images should be placed under images/ in the same directory as the index.md for the proposal.
-->

1. CustomersDot API Endpoint

Create a new API endpoint in CDOT to check namespace trial history. https://gitlab.com/gitlab-org/customers-gitlab-com/-/issues/11481

1. The endpoint should accept a list of namespace IDs as input.
2. For each request, the endpoint should return:
   - trialtype with the namespace id's that are eligible for that type.
3. The response should be in a standardized format (e.g., JSON) for easy parsing by GitLab.

json
```
{
  "namespaces": {
    "1": ["ultimatewithgitlabduoenterprise","ultimateonpremiumwithgitlabduoenterprise"],
    "2": ["ultimatewithgitlabduoenterprise","ultimateonpremiumwithgitlabduoenterprise"],
    "3": ["ultimatewithgitlabduoenterprise"]
  },
  "success": true
}
```

2. Caching Mechanism

- Implement a caching system in GitLab to store trial eligibility information for a namespace after a call to CustomersDot for that information.
- Use Redis as the caching backend for consistency with existing GitLab architecture.
- Cache trial eligibility information with a TTL to balance performance and data freshness.
  - Default GitLab setting of 8 hours should be fine here.

3. Trial Application Process

- Update the trial application process to check the new eligibility criteria.
- Integrate the CustomersDot API call and caching mechanism into the process.

4. Cache Invalidation

- Implement a cache invalidation mechanism to ensure data consistency.
- Invalidate the cache when relevant namespace data changes, such as after a plan update or

after applying for a new trial.

Performance Considerations

- The caching mechanism should significantly reduce the load on the CustomersDot API.
- With caching, the GUI response time should not noticeably increase compared to the current solution that queries data from the GitLab database.
- Optimize cache TTL and invalidation strategies to balance data freshness and system performance.

Scalability Considerations

- The caching system should be designed to handle GitLab.com scale for this added use.

Drawbacks

- Increased complexity in the trial eligibility checking process.
- Potential for inconsistencies if cache invalidation is not handled properly.
- May slightly increase the time taken to troubleshoot trial eligibility issues from the GitLab side due to the CustomersDot API calls and checks.
- Testing the trial flow in development may require more setup to test areas where we would call the CustomersDot API.
  Today we merely create the data in the database. Perhaps we can do the same by populating Redis(our cache).

Alternative Solutions

- Do nothing and use the current GitLab database queries.
  - Rejected due to increasing complexity in the trial eligibility rules and duplication in these rules between CustomersDot and GitLab.
- Storing trial history directly in GitLab instead of CustomersDot
  - Rejected due to the need for consistency with existing systems and potential data duplication.
- Not implementing caching.
  - Rejected due to potential performance issues with frequent API calls to CustomersDot.

SECTION: engineering/architecture/design-documents/sast_glas_diff_scan/_index.md

{{< engineering/design-document-header >}}

Summary

This design document outlines the changes to be made to the Security Widget and Security Pipeline Tab to display diff-based scan results from GitLab Advanced SAST (GLAS). Diff-based scanning focuses on analyzing only the files modified in a merge request and files dependent on it up to a configured neighborhood depth, rather than scanning the entire codebase. This approach significantly reduces scan duration while still providing actionable security insights for most use cases.

If you're unfamiliar with any of the terms used, you may want to skip ahead to the GLAS concepts section for clarification.

Motivation

Full security scans can be time-consuming, especially for large codebases. By focusing the analysis on modified files and their immediate dependencies, we can provide faster feedback on security issues in merge requests.

Goals

- Reduce GLAS scan time for MRs
- Clearly indicate in the Security Widget and Security Pipeline tab that the scan is diff-based
- Ensure fixed vulnerabilities are not displayed in the Security Widget, and clearly explain the reasoning to the user
- Target release in 18.3

Non-Goals

- Stateful incremental scanning (planned for future iterations)
- Create UI components to manually trigger full scans (will use CI/CD variables in MVC)
- Support fixed vulnerability detection (will be hidden in MVC)
- Enable users to configure neighborhood depth (consider for future iteration)

Proposal

Diff-based Scanning

There will be no change to the Security Widget or Pipeline Security Tab for full GLAS scans. Diff-based scanning is not a breaking change, as it can be enabled via the SASTPARTIALSCAN CI/CD variable.

In a GLAS diff-based scan

- Enabled by setting CI/CD variables SASTPARTIALSCAN: differential
The GLAS scanner analyzes only the modified files and the files that depend on them (based on the configured neighborhood depth)
- The resulting SAST report includes a partialscanmode field that is set to differential and findings limited to the scanned files
- The Security Widget displays a notice indicating this is a diff-based scan and only shows new and existing findings, excluding fixed vulnerabilities
- The Security Pipeline tab displays a diff-scan notice

Sequence Diagram for Diff-based Scan Process

mermaid
sequenceDiagram
    participant User
    participant GitLabCI as GitLab CI/CD
    participant GLAS as GitLab Advanced SAST

```
    participant Rails as Rails Backend
    participant MR as MR Security Widget Frontend
    participant Pipeline as Security Pipeline Frontend

    User->>GitLabCI: Configure CI to enable GLAS diff scan
    GitLabCI->>GitLabCI: Configure SASTPARTIALSCAN=true and ASTENABLEMRPIPELINES=true
    User->>User: Creates an MR
    User->>GLAS: Diff-based GLAS scan triggered
    GLAS->>GLAS: Get modified files and neighbourhood files
    GLAS->>GLAS: Run scan
    GLAS->>GLAS: Create SAST report with partialscanmode=differential
    GLAS->>Rails: Upload SAST report
    Rails->>Rails: Store scan with diff metadata
    Rails->>Rails: Set empty fixed findings for diff scan
    Rails-->>MR: Security findings with partialscanmode flag
    MR->>MR: Display diff scan notice
    MR->>MR: Display only new and existing vulnerabilities
    MR-->>User: View security findings
    Rails-->>Pipeline: Security findings
    Pipeline->>Pipeline: Display diff scan notice
    Pipeline->>Pipeline: Display only new and existing vulnerabilities
    Pipeline-->>User: View security findings
```

Database Schema

```
mermaid
classDiagram
    class securityscans {
        id: bigint
        createdat: timestamp
        updatedat: timestamp
        buildid: bigint
        scantype: smallint
        partialscanmode: smallint
        info: jsonb
        projectid: bigint
        pipelineid: bigint
        latest: boolean
        status: smallint
        findingspartitionnumber: integer
    }
```

Key Design Decisions

1. Hide Fixed Vulnerabilities

Decision: Do not display fixed vulnerabilities for GLAS diff-based scans

As part of the MVC implementation, fixed vulnerabilities will not be shown in the Security Widget when diff-based scanning is enabled. This limitation will be clearly communicated to users.

Rationale:

- Diff-based scans cannot reliably detect all fixed vulnerabilities.
- See background and reasoning here

2. Maintain Security Approval Functionality

Decision: Maintain existing security approval rules; document limitations

For the MVC, we will maintain existing security approval functionality but provide clear documentation about potential false negatives when using diff-based scanning with approval rules.

Rationale:

- Maintains current workflows without disruption
- Avoids introducing complex feature interactions in the MVC phase
- See background and reasoning here

3. Full Scan Option

Decision: Users need to disable diff-based scanning to trigger a full scan

We initially considered allowing users with diff-based scanning configured to trigger a full scan for specific pipelines. However, this was blocked by the lack of support for setting manual CI variables for MR pipelines. More context in this issue.

Alternative approach:

- Add a button in the Security Widget to trigger a full scan

Design and Implementation Details

GLAS Analyzer changes

1. Update GLAS analyzer to trigger a diff-based scan when the following CI variables are configured:
   - SASTPARTIALSCAN=differential
   - ASTENABLEMRPIPELINES=true

1. GLAS analyzer only analyzes the modified files and neighborhood files based on the default neighborhood depth

1. For a diff-based scan, the GLAS analyzer will set the partialscanmode field in the sast report to differential.

Report schema changes

1. Update the security report schemas to support a new optional partialscanmode enum field with the value differential for GLAS diff-based scans.

- Since not all analyzers support partial scan modes, this field should be optional.
Reports that don't include it can be assumed to have a null value.
- When incremental scanning is introduced, a new enum value of incremental can be added.

Database Schema Changes

- Add a new field partialscanmode to the securityscans table where the default is null.
- Update the Security::Scan model to define a new enum for the partialscanmode field, with support for the differential value.

Persist the diff-based scan

1. Add partial scan data to the security report parser

1. Create the partial scan metadata in StoreScanService

Prepare data for frontend

MR Security Widget

1. Create a new graphql query that returns enabledreports(which refers to the enabled scanners) as well as the partial scan mode.

- See this comment for how the query might look like
- See this comment on how enabledreports is currently passed down from the backend to the frontend

Pipeline Security Tab

1. Update the pipelineSecuritySummary graphql query to include the partial scan mode data.

Frontend Changes

MR Security Widget

1. Update WidgetSecurityReports to call the new graphql query and replace the existing enabledreports data. Also include the new partial scan mode data.

1. Reference design issue and implement new UI.

Pipeline Security Tab

1. Update the SecurityReportsSummary
to retrieve the updated graphql query containing partial scan mode data.

1. Reference design issue and implement new UI.

GLAS concepts

What does "diff" mean in GLAS diff-based scanning?

In GitLab Advanced SAST, the diff isn't a line-by-line comparison like git diff. Instead, it refers to which files were added or modified in the merge request (MR). These are what I refer to as diff files below. Deleted files are excluded from scanning as they no longer exist, so they·re not relevant to the scan.

Understanding Neighborhood Depth

Neighborhood depth determines how many levels of files that depend on the modified files should be scanned.

- depth = 0: only the diff files are scanned
- depth = 1: diff files + files that import (depend on) the diff files
- depth = 2: depth 1 files and files that import those

We don·t scan files imported by the diff files · only the ones that depend on them. The idea is that changes in a file only affect the files that rely on it.

Example

txt
```
fileA:
  imports: fileB
fileB:
  imports: fileC
fileC:
  imports: fileD
fileD:
  imports: fileE
  note: diff file
fileE:
  imports: -
```

With neighborhood depth = 2 and diff file = fileD:

- Scanned files: fileD, fileC, fileB
- Unscanned files: fileA, fileE

Taint signature (definition)

A taint signature represents how potentially untrusted data flows through the program, from a source (like user input) to a sink (like a dangerous operation such as an SQL query).

In diff-based scanning, we only scan modified files and their dependents because only they can

gain or change taint paths. Files that are imported by a modified file don·t change behavior as a result of the import.

Fixed vulnerabilities in diff scanning

Let·s use the same example again to visualize how the security widget would work with GLAS diff-based scanning, now with vulnerability states:

```
txt
fileA:
  imports: fileB
  vuln: vulnA (new)
fileB:
  imports: fileC
  vuln: vulnB (present in target branch, gone in source branch)
fileC:
  imports: -
  note: diff file
fileD:
  imports: -
```

Neighborhood depth = 2. Diff file = fileC.\
Scanned files: fileC, fileB, fileA.\
Unscanned files: fileD.

In this case:

- vulnA appears newly in the source branch.
- vulnB existed in the target but is no longer detected in the source.

The GLAS SAST report of the source branch only detects vulnA.

Current Security widget would display:

- new: vulnA
- fixed vulnB

Security widget with GLAS diff-based scanning should display(which excludes fixed vuln):

- new: vulnA

Reason being that it is complex for diff-based scanning to determine if vulnB was truly fixed (background of why it's complex). So we plan to hide fixed vulnerabilities from the security widget for GLAS diff-based scans to avoid confusing or inaccurate results.

Cross-file vulnerabilities

Cross-file vulnerabilities occur when a vulnerability spans across multiple files · for example, the source of untrusted data might be in one file, while the sink (where the data is

used unsafely) is in another.

txt
fileA:
  imports: fileB
  sink: vulnX
fileB:
  imports: fileC
fileC:
  imports: fileD
  source: vulnX
fileD:
  imports: -

Neighborhood depth = 2. Diff file = fileC.\
Scanned files: fileC, fileB, fileA.\
Unscanned files: fileD.

In this case, vulnX is detected, because the full taint flow from source to sink is included in the scan.

But if fileD is the diff file:

txt
fileA:
  imports: fileB
  sink: vulnX
fileB:
  imports: fileC
fileC:
  imports: fileD
  source: vulnX
fileD:
  imports: -
  note: diff file

Neighborhood depth = 2. Diff file = fileD.\
Scanned files: fileD, fileC, fileB.\
Unscanned files: fileA.

Result: vulnX is not detected, because the scan didn·t reach the sink in fileA.

References

- Spike diff based scanning for advanced SAST
- Faster Advanced SAST: Diff-based scanning in MRs

SECTION: engineering/architecture/design-documents/sast_ide_integration/_index.md

<!--
Before you start:

- Remove comment blocks for sections you've filled in.
  When your blueprint ready for review, all of these comment blocks should be
  removed.

To get started with a blueprint you can use this template to inform you about
what you may want to document in it at the beginning. This content will change
/ evolve as you move forward with the proposal. You are not constrained by the
content in this template. If you have a good idea about what should be in your
blueprint, you can ignore the template, but if you don't know yet what should
be in it, this template might be handy.

- Fill out this file as best you can. At minimum, you should fill in the
  "Summary", and "Motivation" sections. These can be brief and may be a copy
  of issue or epic descriptions if the initiative is already on Product's
  roadmap.
- Create a MR for this blueprint. Assign it to an Architecture Evolution
  Coach (i.e. a Principal+ engineer).
- Merge early and iterate. Avoid getting hung up on specific details and
  instead aim to get the goals of the blueprint clarified and merged quickly.
  The best way to do this is to just start with the high-level sections and fill
  out details incrementally in subsequent MRs.

Just because a blueprint is merged does not mean it is complete or approved.
Any blueprint is a working document and subject to change at any time.

When editing blueprints, aim for tightly-scoped, single-topic MRs to keep
discussions focused. If you disagree with what is already in a document, open a
new MR with suggested changes.

If there are new details that belong in the blueprint, edit the blueprint. Once
a feature has become "implemented", major changes should get new blueprints.

The canonical place for the latest set of instructions (and the likely source
of this file) is here.

Blueprint statuses you can use:

- "proposed"
- "accepted"
- "ongoing"
- "implemented"
- "postponed"
- "rejected"

-->

{{< engineering/design-document-header >}}

Summary

<!--
This section is very important, because very often it is the only section that
will be read by team members. We sometimes call it an "Executive summary",
because executives usually don't have time to read entire document like this.
Focus on writing this section in a way that anyone can understand what it says,
the audience here is everyone: executives, product managers, engineers, wider
community members.

A good summary is probably at least a paragraph in length.
-->

Support developers performing API-based Static Analysis Security Testing
from their IDE.

Motivation

<!--
This section is for explicitly listing the motivation, goals and non-goals of
this blueprint. Describe why the change is important, all the opportunities,
and the benefits to users.

The motivation section can optionally provide links to issues that demonstrate
interest in a blueprint within the wider GitLab community. Links to
documentation for competing products and services is also encouraged in cases
where they demonstrate clear gaps in the functionality GitLab provides.

For concrete proposals we recommend laying out goals and non-goals explicitly,
but this section may be framed in terms of problem statements, challenges, or
opportunities. The latter may be a more suitable framework in cases where the
problem is not well-defined or design details not yet established.
-->

Goals

What is it trying to achieve?

- Provide SAST results to Ultimate users running any GitLab Editor Extension.

How will we know that this has succeeded?

- Users with the required connectivity receive diagnostics for SAST findings.

What are other less tangible opportunities here?

- Defining how non-SAST security scan results may be presented in the IDE in the future.
- Populate IDE diagnostics from an existing SAST report.

Non-Goals

What is out of scope for this blueprint?

- Defining how offline users will run analyzers locally.

Proposal

Provide both local and API-based security scans against the current IDE workspace.

Pros:

- All platforms where we have GitLab Editor Extensions would support SAST findings.
- Scan service API could be implemented through a local service to consume existing local SAST reports.
- Offline users could use offline scan images or distributions.

Cons:

- We must deploy new infrastructure.
- We must be intentional in showing/hiding this feature based on the user, group, and project configuration.

Decisions

- 001: Provide API-based security scans

Design and implementation details

<!--
This section should contain enough information that the specifics of your
change are understandable. This may include API specs (though not always
required) or even code snippets. If there's any ambiguity about HOW your
proposal will be implemented, this is the place to discuss them.

If you are not sure how many implementation details you should include in the
blueprint, the rule of thumb here is to provide enough context for people to
understand the proposal. As you move forward with the implementation, you may
need to add more implementation details to the blueprint, as those may become
an important context for important technical decisions made along the way. A
blueprint is also a register of such technical decisions. If a technical
decision requires additional context before it can be made, you probably should
document this context in a blueprint. If it is a small technical decision that
can be made in a merge request by an author and a maintainer, you probably do
not need to document it here. The impact a technical decision will have is
another helpful information - if a technical decision is very impactful,
documenting it, along with associated implementation details, is advisable.

If it's helpful to include workflow diagrams or any other related images.
Diagrams authored in GitLab flavored markdown are preferred. In cases where
that is not feasible, images should be placed under images/ in the same
directory as the index.md for the proposal.
-->

Remote scans

mermaid
sequenceDiagram
    actor User

    User->>+IntelliJ: Edits a file "hello.rb"
    User->>+IntelliJ: Saves "hello.rb"
    IntelliJ->>+GitLab Duo for JetBrains: Triggers a security scan based on the current file and workspace
    GitLab Duo for JetBrains->>+GitLab API: POST /api/v4/securityscans/sast
    note over GitLab Duo for JetBrains,GitLab API: { "path": "hello.rb", "content": "exec(ARGV[0])" }
    GitLab API->>+Security Scan Service: POST /v1/sast
    Security Scan Service->>-GitLab API: 200 OK
    GitLab API->>-GitLab Duo for JetBrains: 200 OK
    GitLab Duo for JetBrains->>-IntelliJ: Display Static Analysis findings (Diagnostics)

Alternative Solutions

Do nothing

The current experience for GitLab users in the IDE is that they must run separate
Static Analysis tools locally before pushing their code and waiting on their CI/CD
pipeline's security scan results.

Run analyzers locally as offline analyzer

Pros:

- Local data and execution simplifies performance optimization
- A narrower usecase allows for a simpler, more tightly coupled design

Cons:

- We would need to start supporting binary distributions along with a release cycle that limits our ability to distribute rule refinements and bugfixes
- We would need to codesign our binaries especially for Mac OS.
- We would need to provide documentation for installation.
- We would need to provide tooling for installation.

SECTION: engineering/architecture/design-documents/sast_ide_integration/decisions/001_provide_api-based_security_scans.md

Summary

1. The original spikes for SAST integration explored two avenues: local scanning or remote scanning
1. A number of tradeoffs were considered with the focus was on iterating towards an effective integration with a fixed timeline

A remote scanning solution supports a larger usecase, faster onboarding and it potentially reduces maintenance effort while increasing observability in comparison to a local scanning solution.

Initially, we plan to mitigate the costs of remote data access by scanning on a per-file basis. By minimizing the time to market, we can more quickly begin iterating on features according to customer demand.

Context

Summarized from the original spike discussion:

As covered in the epic's technical needs we want as few installation steps and dependencies as possible; a goal that is most easily achieved by hosted, remote scanning solutions. It also fits GitLab's existing deployment paradigms and evolving best practices around Project Runway and the AI Gateway which already provide pathways to realise service integrations. In contrast, for local scanning we do not have these pathways yet.

Both the spike and previous technical discovery assumed we must retain ephemeral environments (modeling our CI builds), so we maintained this constraint.

The main benefits of a remotely hosted solution:

- rapid customer onboarding (see above)
- continuous deployments (shipping binary ties us to a limited release cycle and longer support windows for those who dont upgrade, both for rule refinement and bugfixes)
- observability and centralized telemetry
- reduced costs incurred for customers
- leveraging service for platform usecases beyond IDE code search (best example is matching this paradigm for platform-wide secret detection; i.e. scanning MR descriptions for secrets) or running analysis on-demand

Future Considerations

We will focus on UX constraints, IDE integration, and coordinated deployment from the starting point of an entirely remote, single-file scanner.

With each usability question and performance concern, we can redress execution and data locality in the appropriate context of a functioning system.

SECTION: engineering/architecture/design-documents/secret_detection/_index.md

<!-- vale gitlab.FutureTense = NO -->
{{< engineering/design-document-header >}}

Summary

Today secret detection focuses on scanning repositories in a pipeline. We aim to broaden the scope of Secret Detection to cover more areas where leaks or compromised tokens might surface.

Evolving secret detection into a comprehensive, platform-wide experience, will extend coverage to high-risk areas. We will expand the secret detection feature set to detect secrets before they are pushed, in job logs, and in issues, epics, and merge requests.

Motivation

Goals

- Support platform-wide detection of tokens to avoid secret leaks
- Prevent exposure by rejecting detected secrets
- Provide scalable means of detection without harming end user experience
- Unified list of token patterns and masking

See target types for scan target priorities.

Non-Goals

Phase1 is limited to detection and alerting across platform, with rejection only during prereceive Git interactions and browser-based detection.

Secret revocation and rotation is also beyond the scope of this new capability.

Scanned object types